Requirement Management and Agile Software Development

Shola Oyedeji

Characteristics of Excellent Requirements

1. **Complete.** Each requirement must fully describe the functionality to be delivered. It must contain all the information necessary for the developer to design and implement that bit of functionality.

2. **Correct.** Each requirement must accurately describe the functionality to be built. A software requirement that conflicts with its parent system requirement is not correct. Only user representatives can determine the correctness of user requirements, which is why users or their close surrogates must review the requirements.

3. **Feasible.** It must be possible to implement each requirement within the known capabilities and limitations of the system and its operating environment. To avoid specifying unattainable requirements, have a developer work with marketing or the requirements analysts throughout the elicitation process.

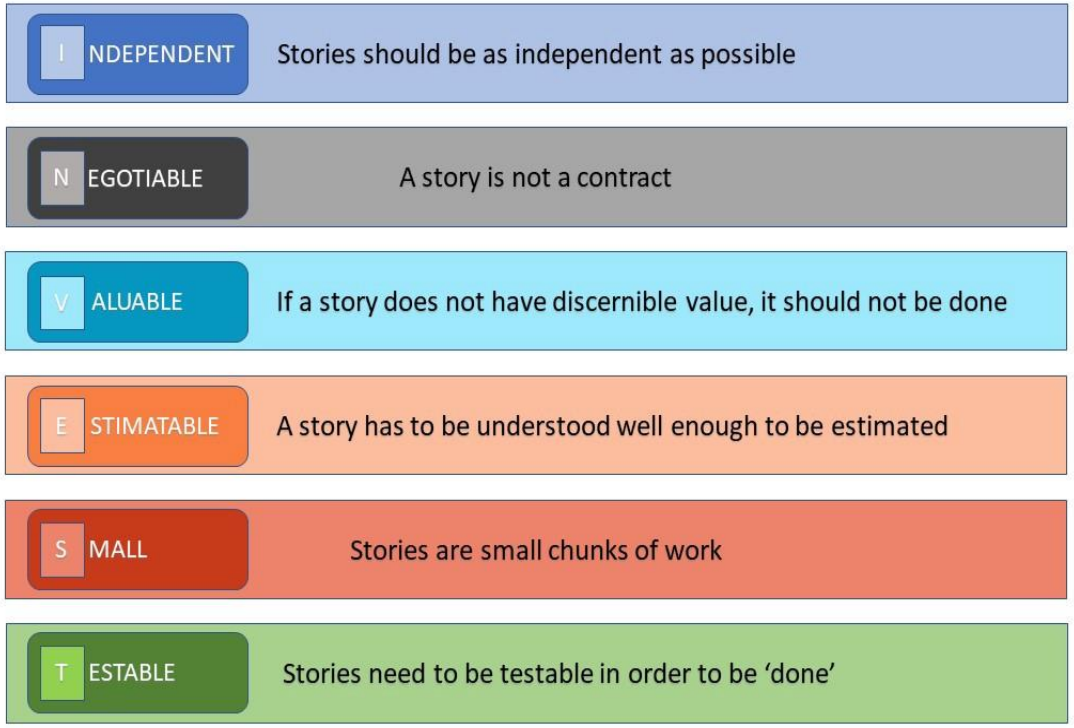
4. **Necessary.** Each requirement should document a capability that the customers really need or one that's requirement for conformance to an external system requirement or a standard. Every requirement should originate from a source that has been authority to specify requirements. Trace each requirement back to specific voice-of-the-customer input.

5. **Prioritized.** Assign an implementation priority to each functional requirement feature to indicate how essential it is to a particular product release. If all the requirements are considered equally important, it is hard for the project manager to respond to budget cuts, schedule overruns, personnel losses, or new requirements added during development

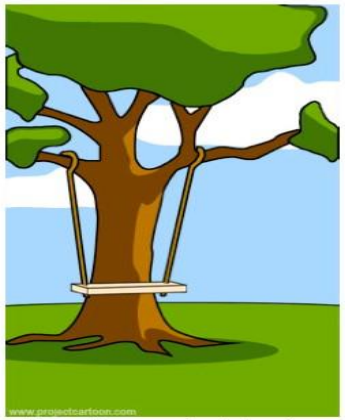
User Story

A good user story must follow the **INVEST** principle:

- **Independent**: Each user story should stand on its own, independent from other stories.
- **Negotiable**: Stories are not contracts, rather opportunities for negotiation and change.
- **Valuable**: Every story should add value for users and stakeholders.
- **Estimable**: Every story's time and budget costs should be calculable, based on domain and technical knowledge.
- **Small**: User stories should be small enough to estimate and implement simply.
- **Testable**: Make sure you can test the user story through criteria the story itself explains.



How the customer explained it



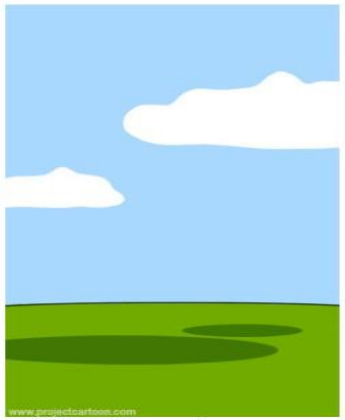
How the project leader understood it

How the analyst designed it

How the programmer wrote it

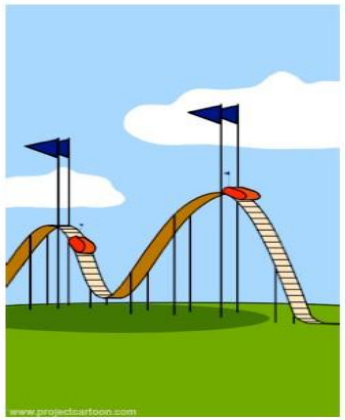
What the beta testers received

How the business consultant described it

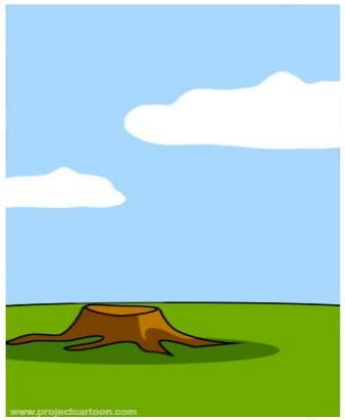


How the project was documented

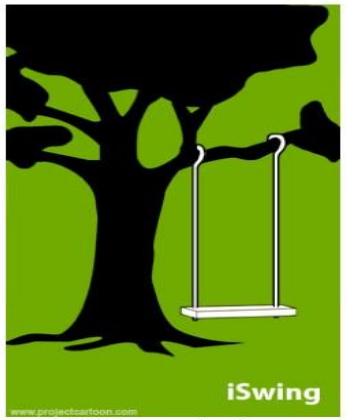
What operations installed



How the customer was billed



How it was supported



What marketing advertised

What the customer really needed

Agile Requirement and Software Development

Agile Umbrella

Scrum
Lean software development
Kanban (process + method)
Extreme Programming (**XP**)
Continuous Integration (**CI**)
Continuous Delivery (**CD**)
Feature Driven development (**FDD**)
Test Driven Development (**TDD**)
Crystal Clear
...

Lightweight approaches

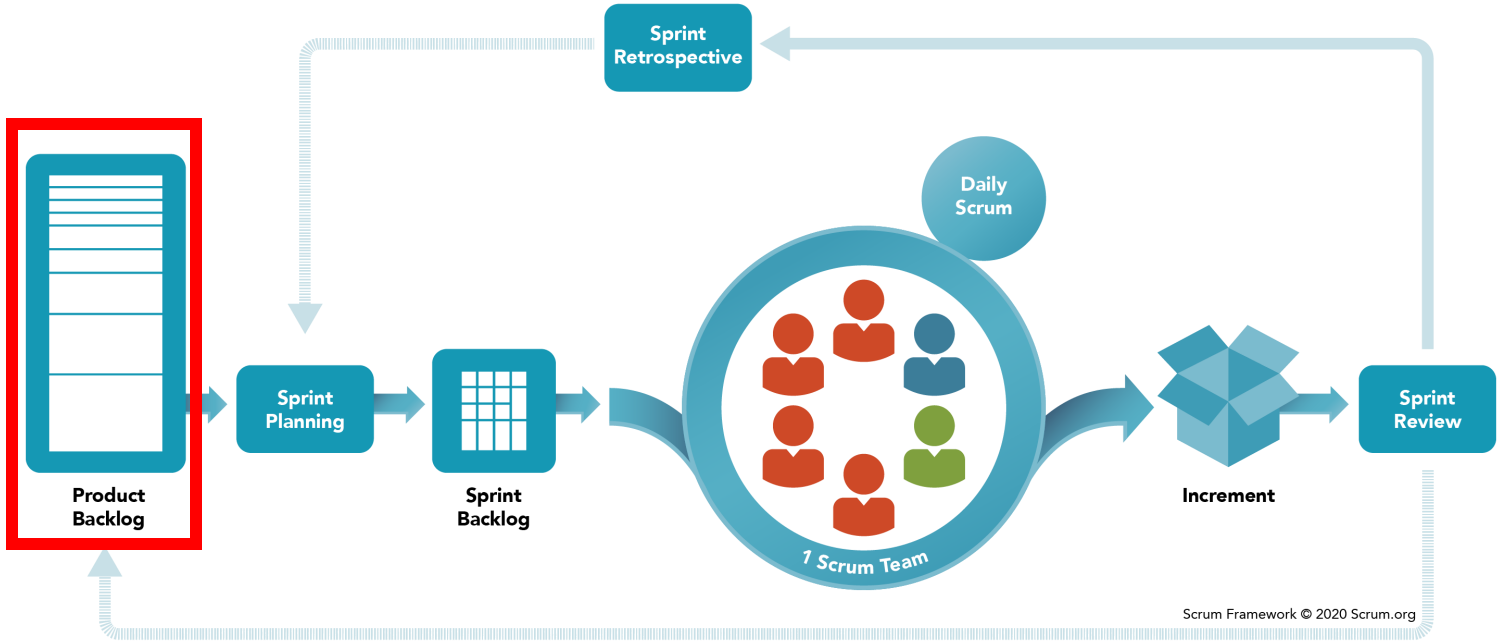
Scrum-of-Scrums
Scrum at Scale (**Scrum@Scale**)
Large-scale Scrum (**LeSS**)
Scaled Agile Framework (**SAFe**)
Disciplined Agile Delivery (**DAD**)
Dynamic Systems Development Method (**DSDM**)
Agile Project Management (**AgilePM**)
Agile Unified Process (**AUP**)
Open Unified Process (**OpenUP**)
...

Fuller approaches (beyond 1 team)

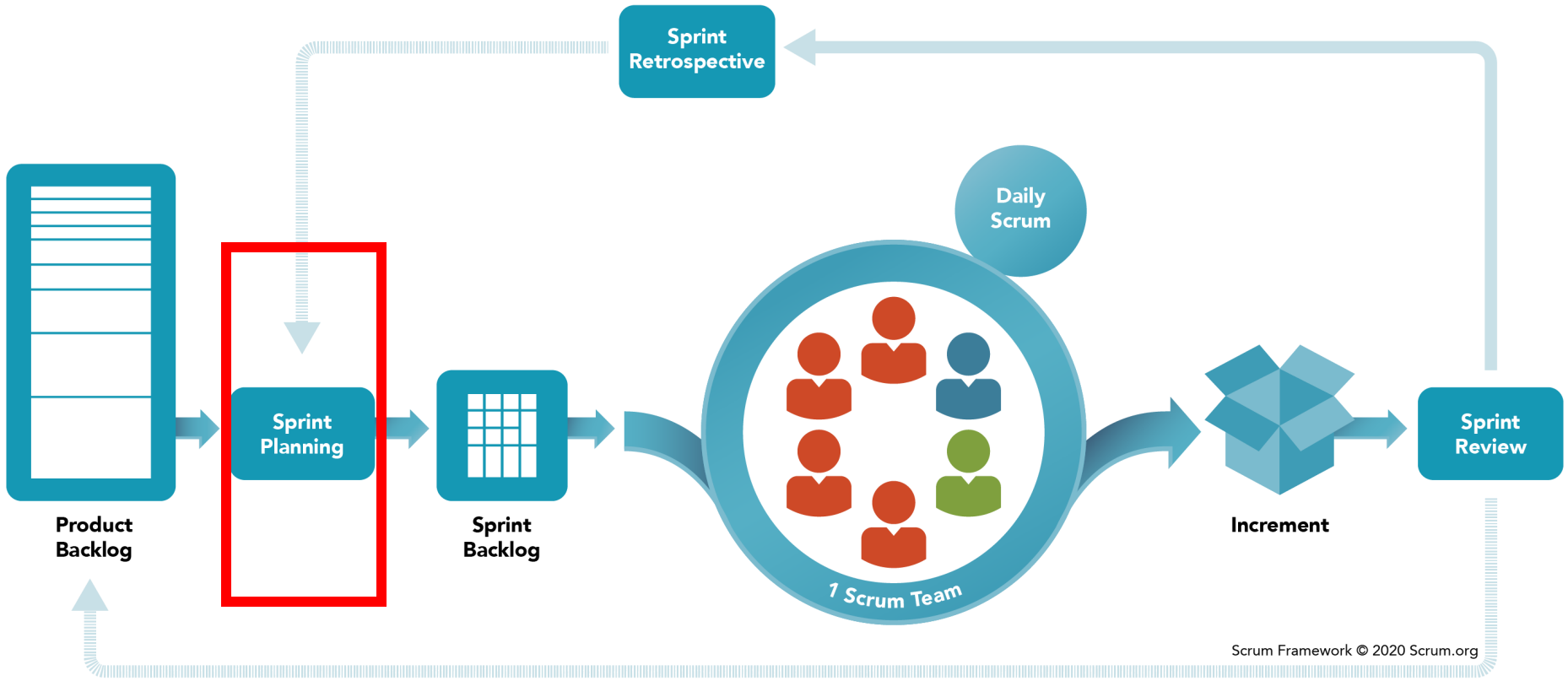
12 Agile Principles

Scrum

- Scrum is one of the most famous agile methodologies characterized by cycles or stages of development called sprints.
- Scrum is a framework used by people to address complex adaptive problems



Scrum – Sprint Planning



Sprint Planning

Story point is a unit of measurement to estimate the effort, complexity, and uncertainty of a user story or task

T-shirt sizing is used in to quickly and intuitively estimate the effort, complexity, or uncertainty of a user story or task.

- **XS (Extra Small)** – Very simple, minimal effort.
  S (Small) – Straightforward task, low complexity.
  M (Medium) – Moderate complexity, requires some effort.
  L (Large) – More complex, may take longer.
  XL (Extra Large) – Highly complex, likely to be broken down further.
  XXL (Too Big) – Too large for a sprint, must be split into smaller stories.

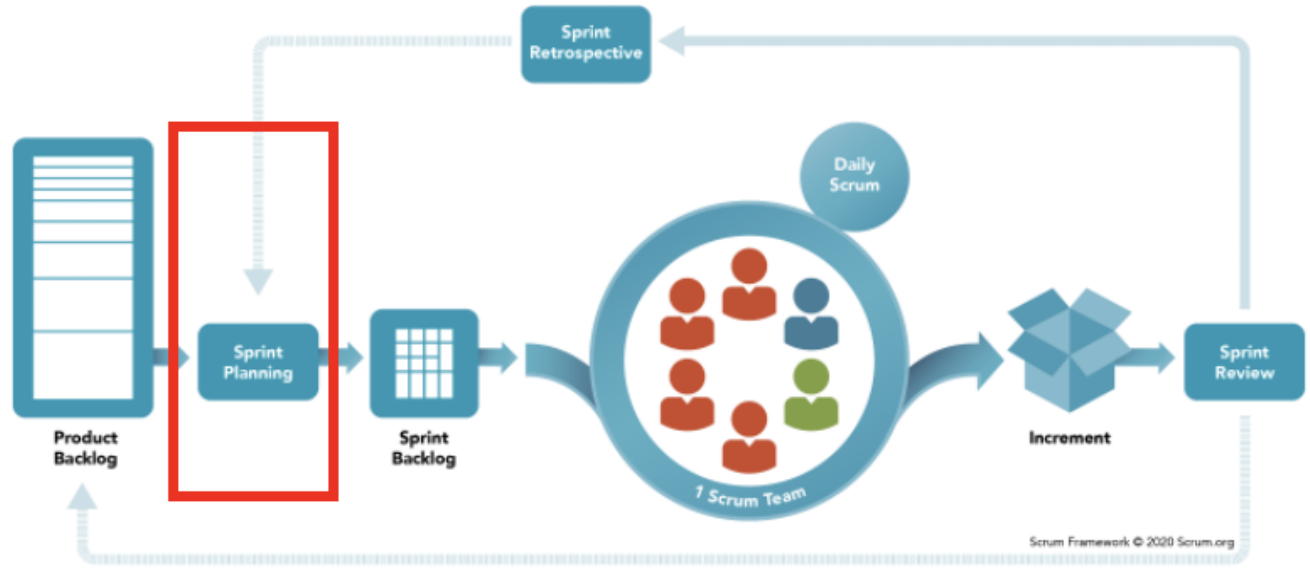
T-Shirt Size	Story Points
XS	1
S	2
M	3 - 5
L	8
XL	13 - 21 (Too big, break it down)

User Story	Story Points
"As a user, I want to reset my password via email."	1
"As a user, I want to search for tasks based on location and category."	3
"As a task poster, I want to securely pay Taskers through the app."	5
"As a task poster, I want to manage disputes and request refunds."	8 (Break into smaller stories)

Sprint Planning

Backlog refinement: Prioritize and break down tasks.

- Effort estimation: story point, T-shirt sizing.
- Sprint Goal Definition: Focus on key deliverables.
- Organizing Sprint Backlog: Select the highest priority items.



Example Sprint Goal: Implement user authentication

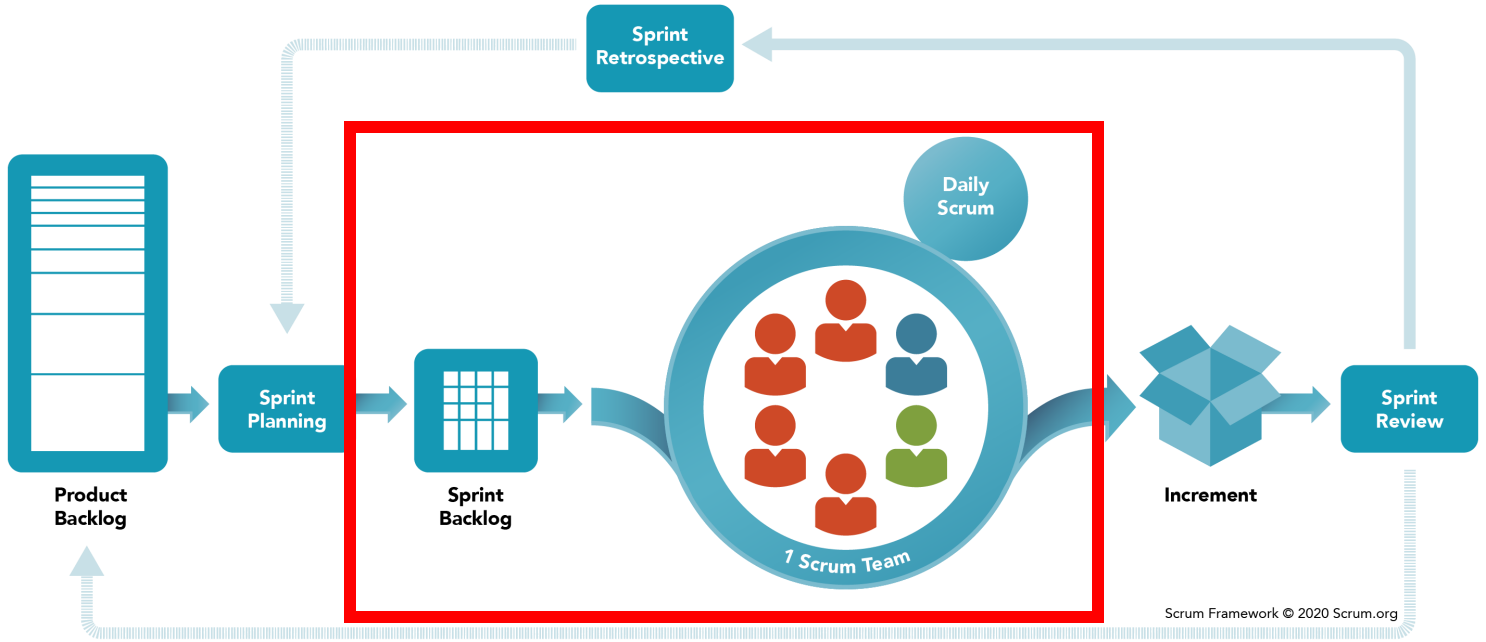
Scrum

Sprint Backlog

- To-do list for the Sprint.
- Contains all the tasks, user stories, and goals that the team commits to completing during the current Sprint.

Sprint

- A short and fixed period, usually 1 to 4 weeks
- The development team works to complete a specific set of tasks or features.
- The team plans, builds, tests, and delivers a small, usable part of the product.



AGILE REQUIREMENT PROCESS

1. Kick-off or Requirement Workshop

Goal: Create a shared understanding among stakeholders, Product Owner (PO) and teams (analyst, requirement engineer, developer, user experience (UX) expert).

- Conduct user or customer interviews to understand your customers' pain points, motivations, and expectations.

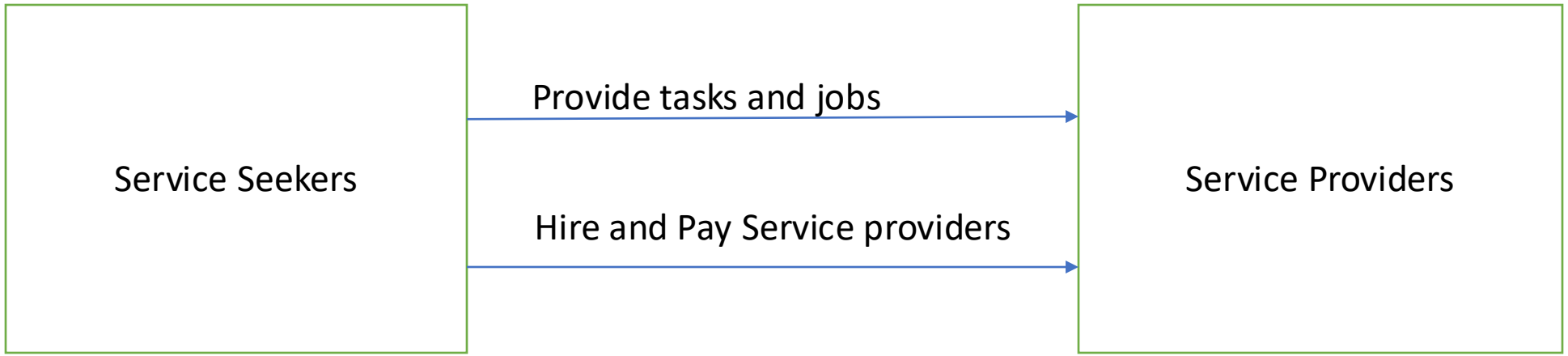
How?

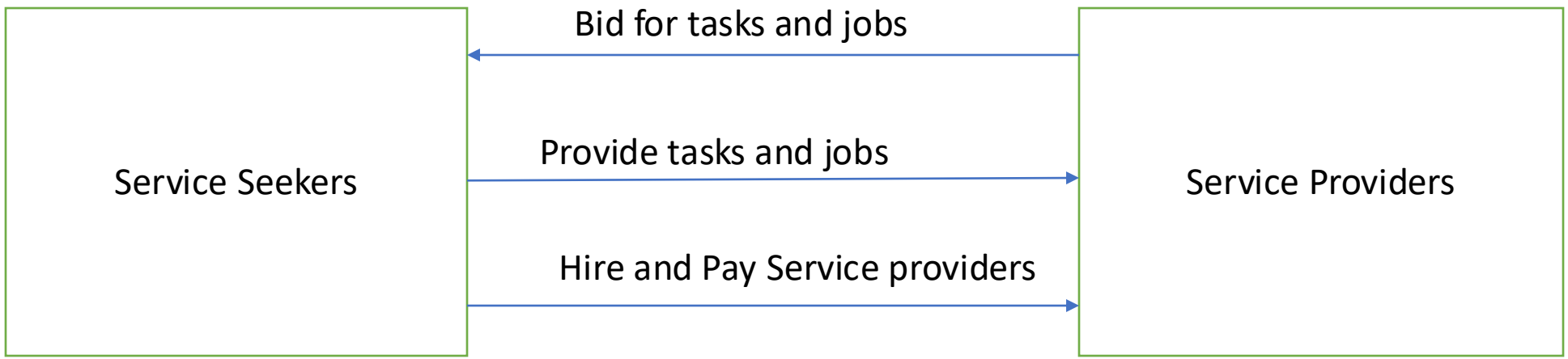
1. Identify a diverse group of users who represent various perspectives, roles, and levels of expertise both from your side and the customer side.
2. Prepare a list of open-ended questions that focus on users' experiences, needs, pain points, and expectations
3. Interactive prototyping: used to gather high-level feedback
4. Record and analyze the information collected during the interviews, identifying common themes and trends that can help inform the project's requirements.

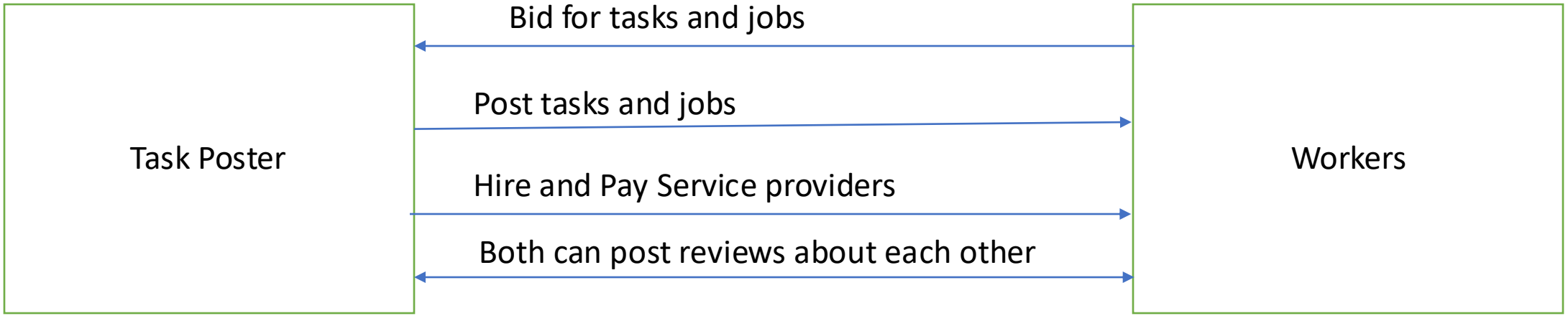
Next: Translate identified user needs into user stories

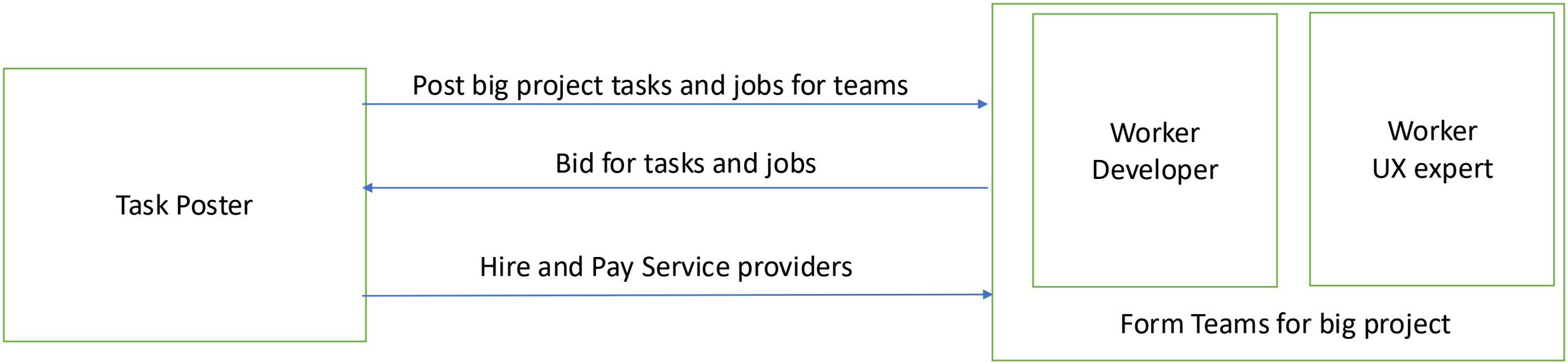
Example

- I need you to create an online marketplace that connects people who need help with various everyday tasks to skilled workers. I want a platform that allows people to post tasks, hire freelancers for short-term jobs, and facilitate seamless transactions.









Example

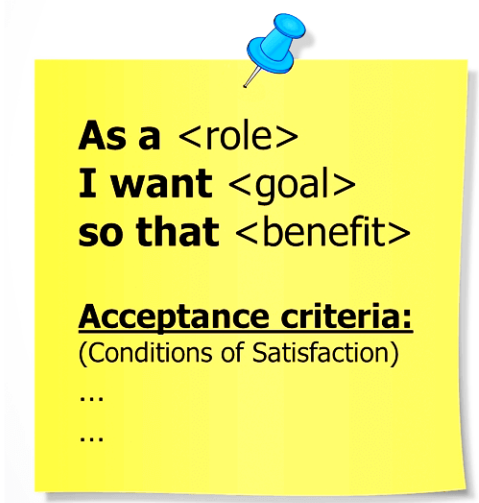
I need you to create an online marketplace that connects people who need help with various everyday tasks to skilled workers. I want a platform that allows people to post tasks, hire freelancers for short-term jobs, and facilitate seamless transactions.

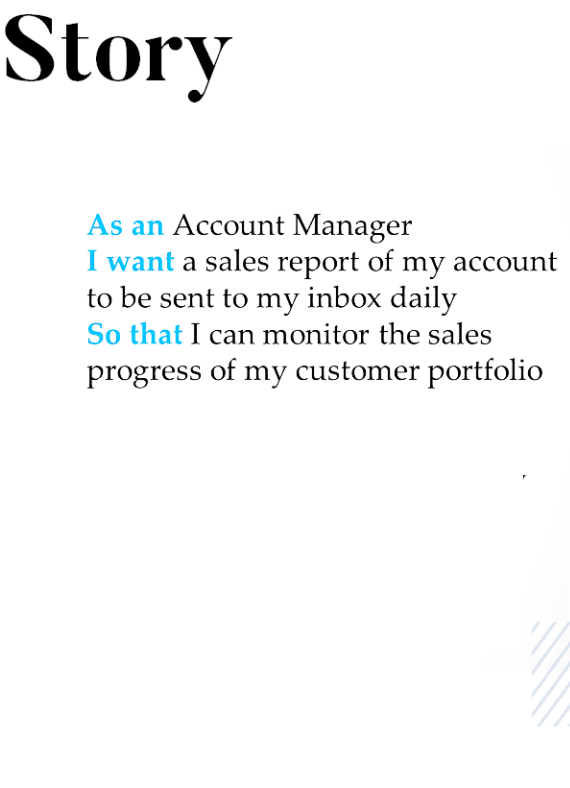
- **Task Posting:** create task listings, job details, and set budgets
- **Task Matching:** The platform suggests qualified workers based on skills and location
- **Bidding System:** Service providers can apply for tasks and negotiate pricing.
- **Secure Payments:** Transactions are processed securely within the platform.
- **Ratings and Reviews:** Users can provide feedback on completed tasks.
- **Chat and Notifications:** In-app messaging allows seamless communication between service seekers (task posters) and providers.

User Story

Example of user story:

- Structure
- Acceptance Criteria





Source

User Story

Example of user story:

- Acceptance Criteria

User story: *As a user, I want to use a search field to type a city, name, or street, so that I can find matching hotel options.*

Acceptance criteria

- The search field is placed on the top bar
- Search starts once the user clicks “Search”
- The field contains a placeholder with a grey-colored text: “Where are you going?”
- The placeholder disappears once the user starts typing
- Search is performed if a user types in a city, hotel name, street, or all combined
- Search is in English, French, German, and Ukrainian
- The user can not type more than 200 symbols

Source

Epics in Agile Software Development

Epic is a large user story that represents a high-level feature, objective, or requirement within a project.

- User Registration
- Task Posting
- Task Matching
- Payment
- Conflict Resolution

Epic: User Registration and Authentication (Must-Have)

- User Story 1: As a new user, I want to sign up with email and password, so that I can create an account and access services.
- User Story 2: As a user, I want to log in using my email and password, so that I can access my profile securely.
- User Story 3: As a user, I want to log in using Google/Facebook authentication, so that I can quickly access my account.
- User Story 4: As a user, I want to reset my password via email, so that I can regain access if I forget my password.

EPIC: Task Posting and Management (Must-Have)

- User Story 5: As a task poster, I want to post a task with a description, category, and budget, so that I can find workers to complete it.
- User Story 6: As a task poster, I want to set a location for my task, so that I can hire workers nearby.
- User Story 7: As a task poster, I want to edit or delete my task, so that I can update or remove unnecessary listings.
- User Story 8: As a task poster, I want to receive notifications when someone applies for my task, so that I can review and respond.

EPIC: Task Searching and Bidding (Must-Have)

- User Story 9: As a worker, I want to search for available tasks by category, location, and budget, so that I can find relevant work.
- User Story 10: As a worker, I want to apply for a task by submitting a proposal, so that I can be considered for the job.
- User Story 11: As a task poster, I want to review worker applications and select a candidate, so that I can assign the task.

EPIC: Payments and Transactions (Must-Have)

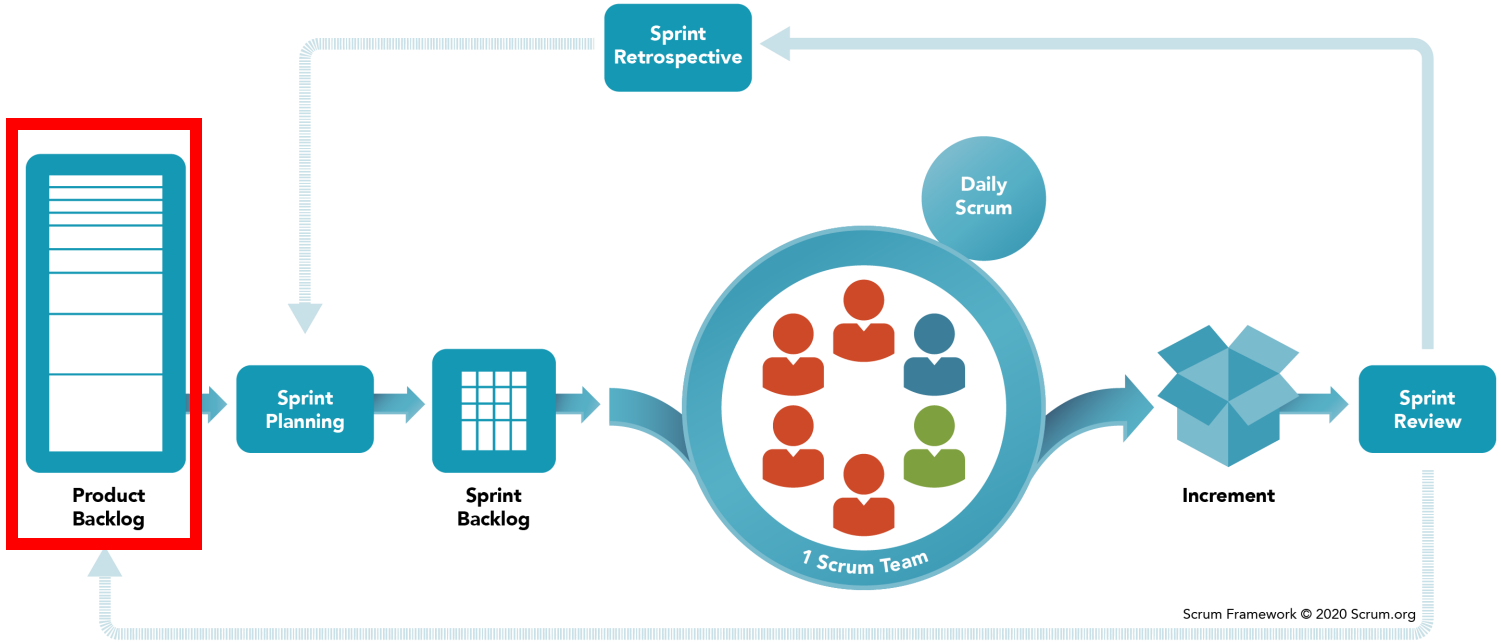
- User Story 12: As a task poster, I want to securely pay for a task via credit card, so that I can ensure the worker gets paid.
- User Story 13: As a worker, I want to receive payments through the platform, so that I get paid securely.
- User Story 14: As a worker, I want to see my payment history, so that I can track my earnings.
- User Story 15: As an admin, I want to handle payment disputes, so that I can resolve issues fairly.

EPIC: Ratings and Reviews (Nice-to-Have)

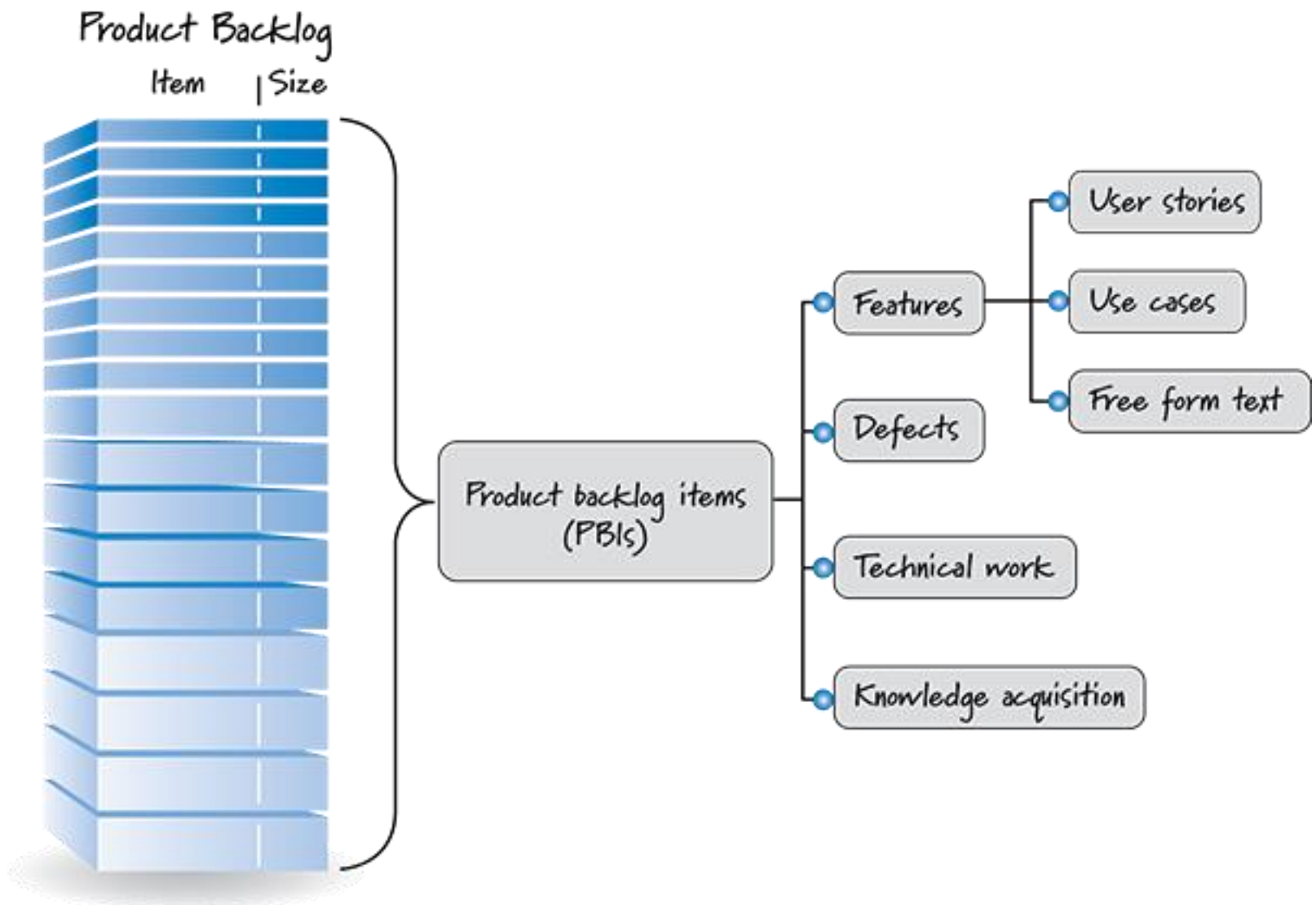
- User Story 16: As a task poster, I want to rate and review a worker, so that I can share feedback for future users.
- User Story 17: As a worker, I want to rate and review task posters, so that I can warn others about unreliable clients.
- User Story 18: As a user, I want to view ratings and reviews before hiring someone, so that I can choose a reliable worker.

Scrum

- Scrum is one of the most famous agile methodologies characterized by cycles or stages of development called sprints.
- Scrum is a framework used by people to address complex adaptive problems



Scrum: Product Backlog



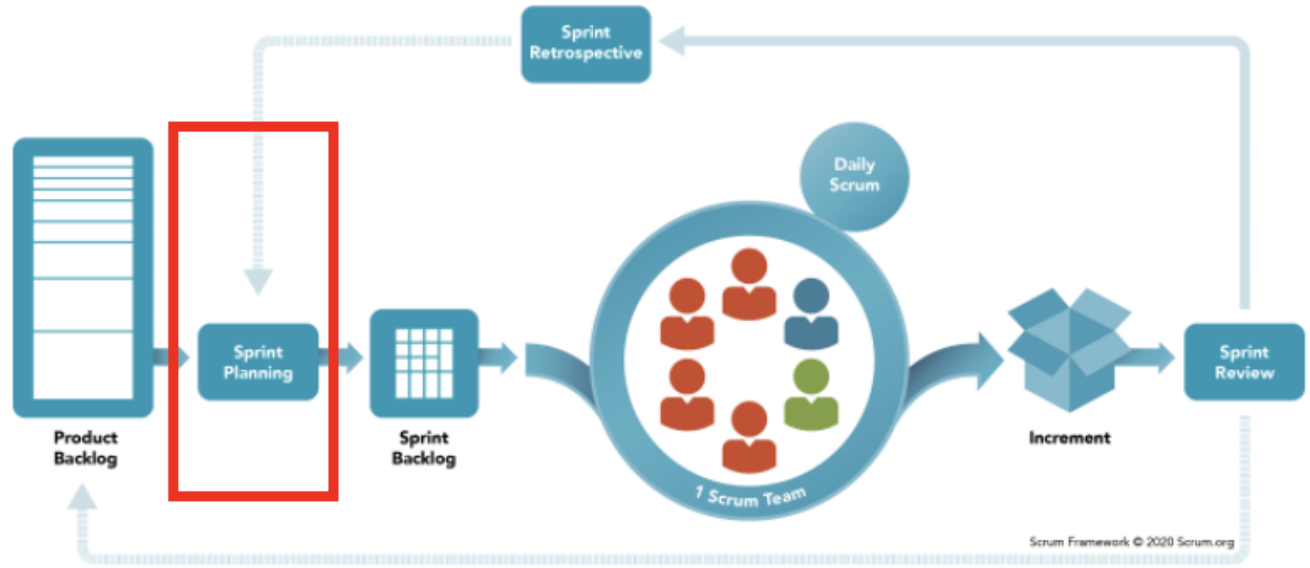
Scrum Backlog

Epic	User Story ID	User Story Description	Priority (MoSCoW)	Acceptance Criteria
User Registration and Authentication	US1	As a new user, I want to sign up with email and password so that I can create an account.	Must Have	Account is created; confirmation email received.
	US2	As a user, I want to log in using email and password so that I can securely access my profile.	Must Have	Successful login redirects to dashboard.
	US3	As a user, I want to log in using Google/Facebook so that I can quickly access my account.	Should Have	OAuth login works; existing users can link accounts.
	US4	As a user, I want to reset my password via email so that I can regain access if I forget my password.	Must Have	Password reset email is sent; new password works.

Sprint Planning

Backlog refinement: Prioritize and break down tasks.

- Effort estimation: story point, T-shirt sizing.
- Sprint Goal Definition: Focus on key deliverables.
- Organizing Sprint Backlog: Select the highest priority items.



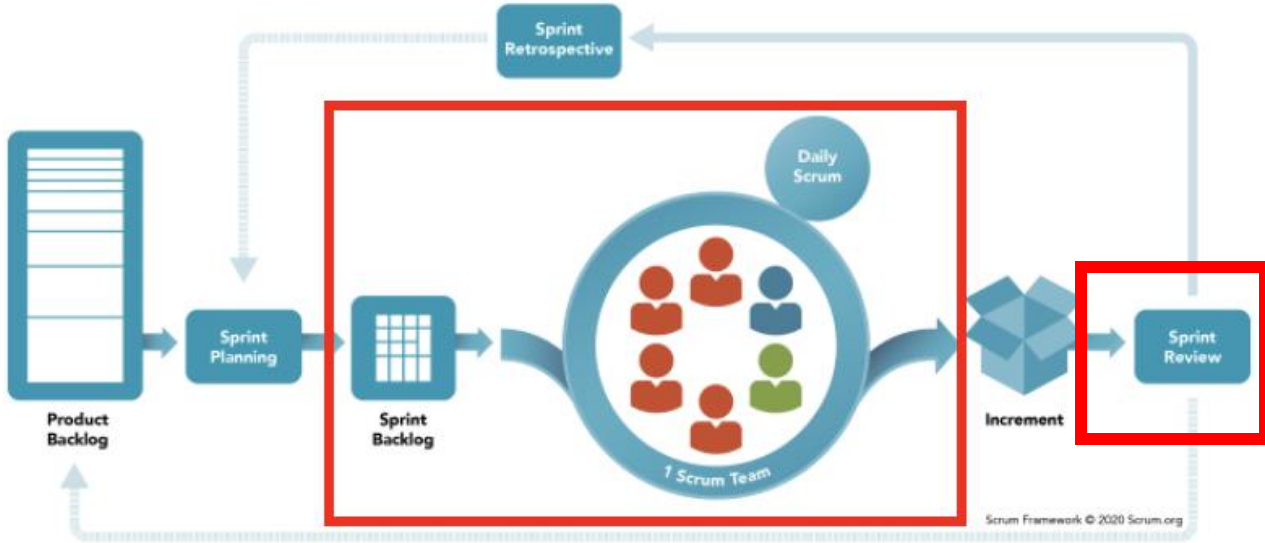
Example Sprint Goal: Implement user authentication

Example: Sprint 1 Execution

Sprint Goal: Implement user authentication.

Selected Backlog Items:

- User registration (email/password, OAuth)
- Login and Logout functionality
- Password reset feature



Sprint Review:

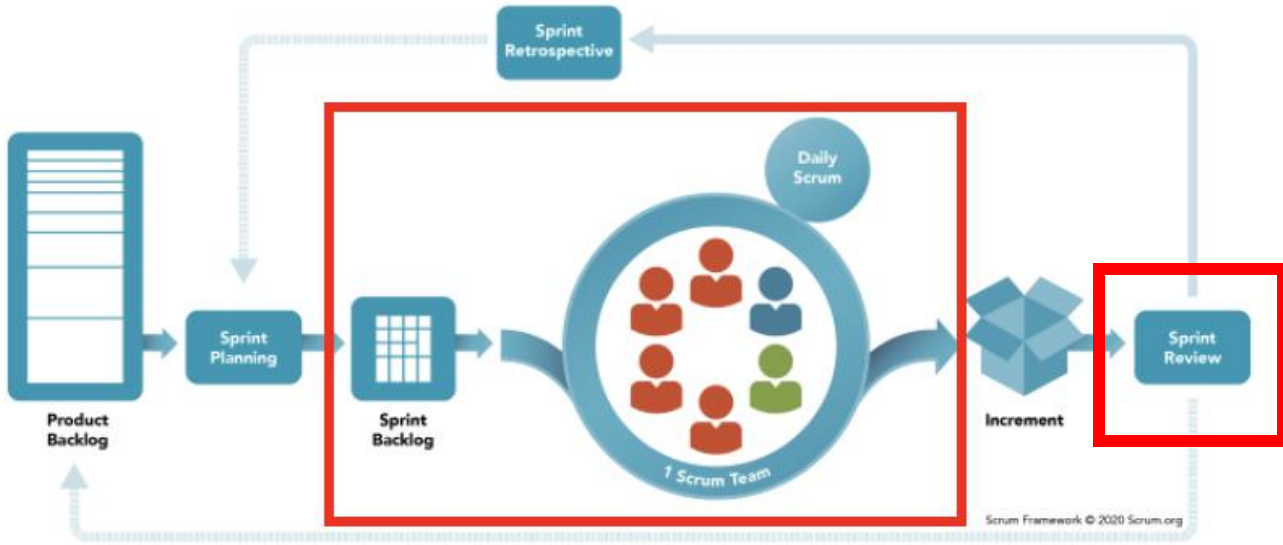
- User testing conducted
- Adjustments made to UI based on feedback

Example: Sprint 2 Execution

Sprint Goal: Task posting and matching.

Selected Backlog Items:

- Allow users to post tasks
- Implement task search and filtering
- Notification system for new task assignments



Sprint Review:

- Users provided feedback on search accuracy
- Adjustments made to search algorithm

Sprint Retrospective for Both Sprints

What Went Well:

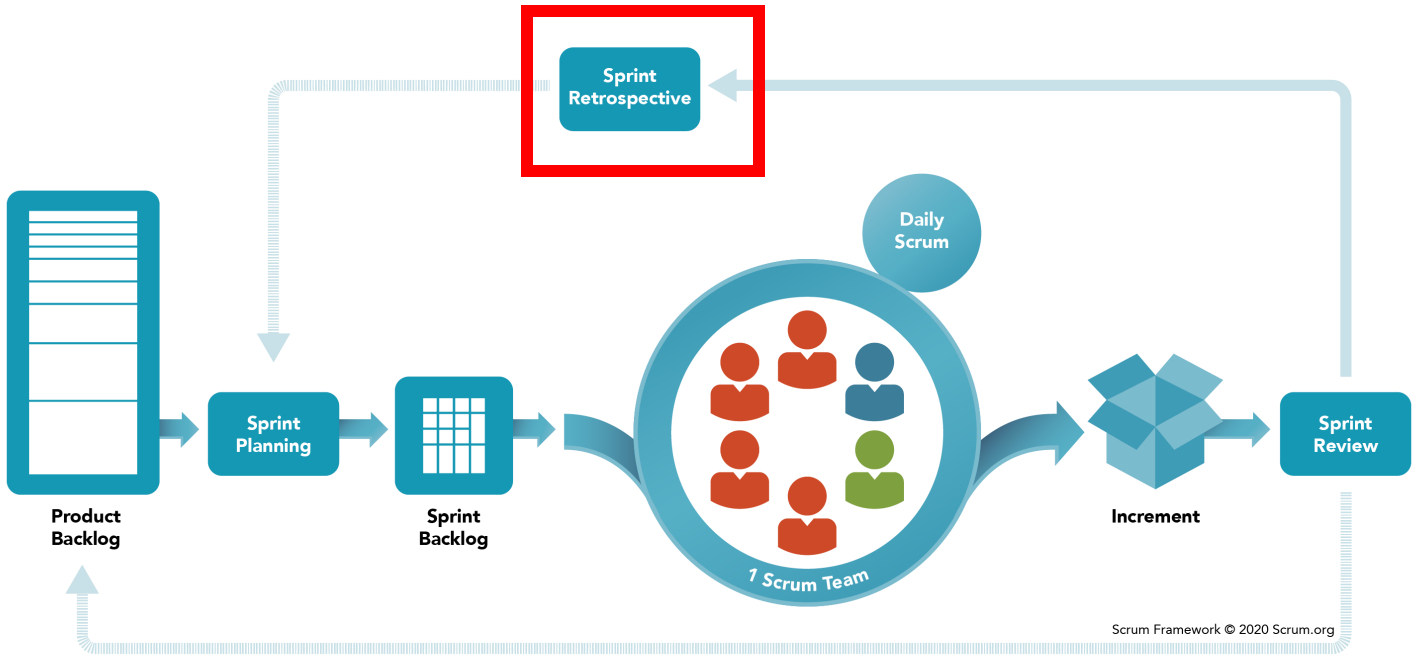
- User registration and authentication tested successfully.
- Task posting received positive feedback.

What Did not Go Well:

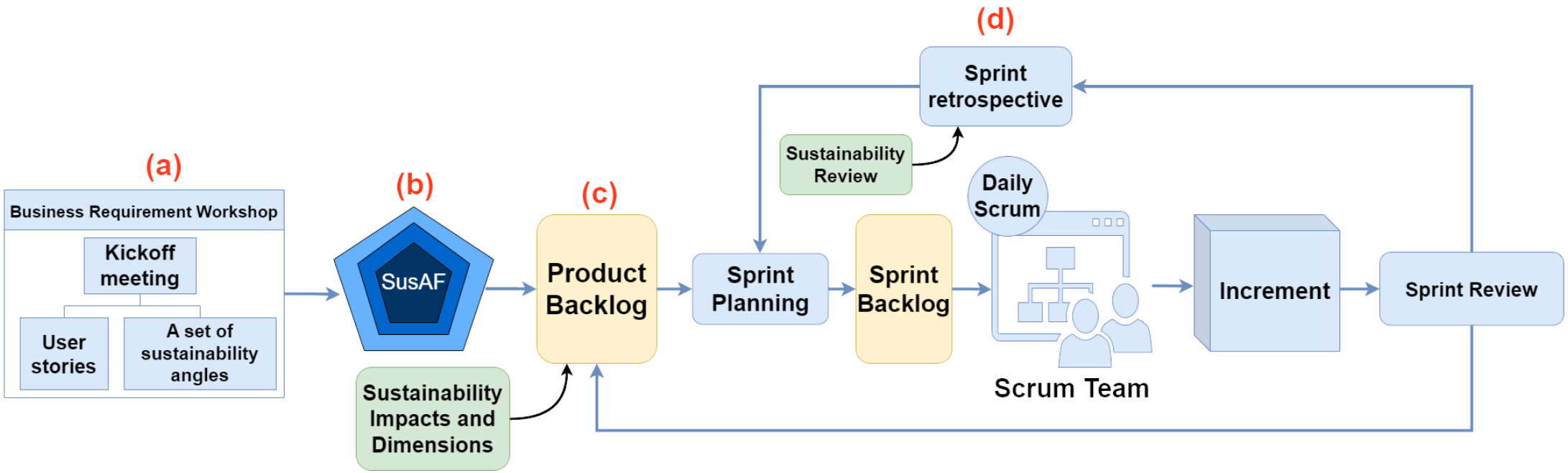
- Search functionality needs refinement.

Improvements:

- Enhance the search algorithm.
- Improve UI responsiveness.



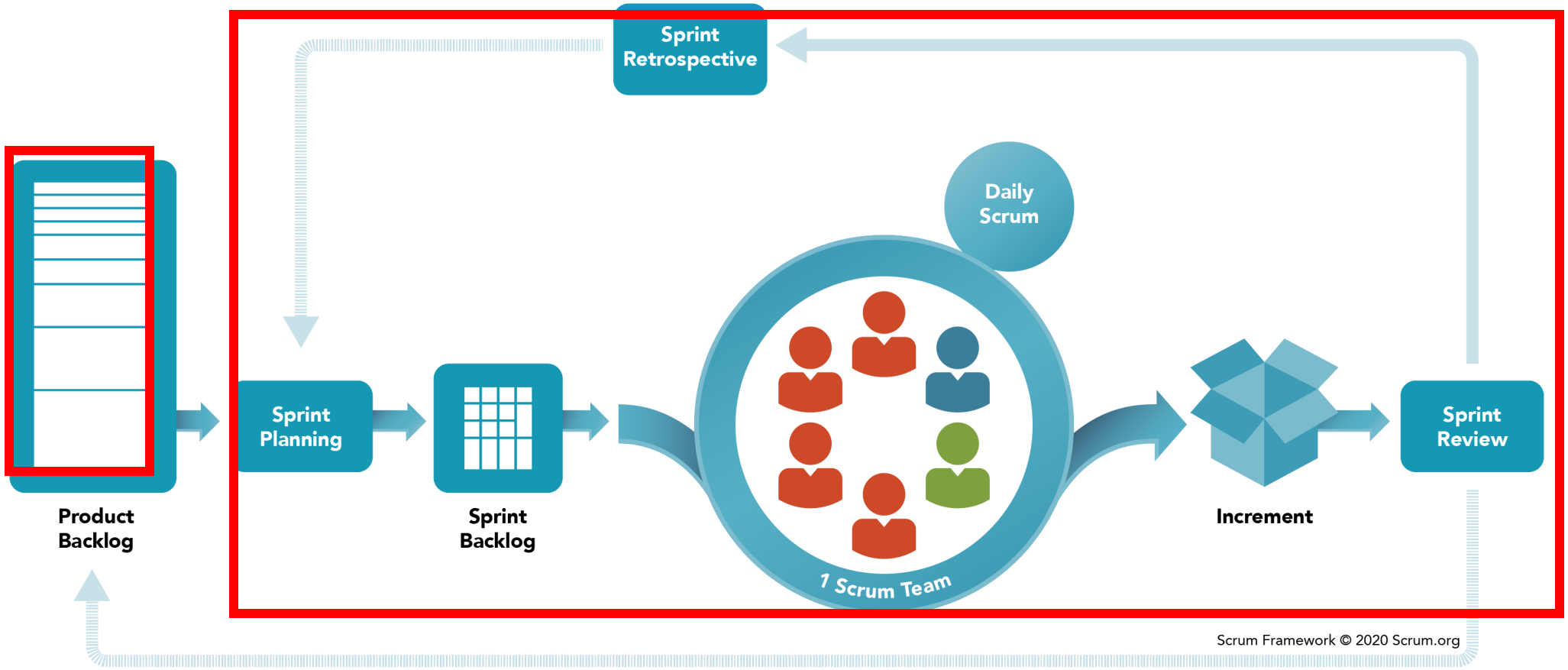
How can you redesign the scrum product backlog with sustainability in mind?



Sprint Retrospective

Sprint	OBSERVING: What worked best and helped sustainability design decisions	OBSERVING: What were the obstacles and challenges	REFLECTING: What can we do differently to improve the overall sprint process?	PLANNING: What should be incorporated in the next sprint?
Sprint 1	One developer played the role of sustainability rep to keep the team on track as they made design decisions during this sprint.	It was challenging to use Jira and Excel templates at the same time. Team lacked sustainability knowledge and relied on SusAF, templates, and the researchers.	Incorporate the sustainability impacts into the sprint backlog. Establish some metrics to measure the sustainability of each sprint outcome.	Add sustainability impacts the sprint backlog. The researcher should join our sprint planning as a sustainability stakeholder.
Sprint 2	The continuous refactoring sessions helped reduce technical debt and keep the codebase clean. The cost-benefit analysis was useful to prioritize features that deliver maximum value	Despite efforts to optimize, the software product energy consumption is still higher than the desired consumption, especially during testing phases.	Identify and use better optimization strategies to improve the overall software product energy consumption.	Target reduction of energy consumption by 15% in the next sprint. Increase efforts to ensure diverse team participation in decision-making.
Sprint 3	The optimization of CI/CD pipelines reduced the energy consumption for the build and deployment processes.	Rapid iterations forced fast development and release, risking inefficient code results. This could cause technical debt and affect future maintainability	Create a better plan for each sprint iteration with a focus on supporting the team to create quality codes and test cases.	Work on efficient resource allocation to reduce delays in certain tasks that affect cost-effectiveness.
Sprint 4	The implementation of a modular architecture is good, which will aid in better long-term maintainability and adaptability in subsequent development iterations.	For individual sustainability, there is still a lack of personalization features, which might affect the ability of the software to meet the diverse needs of all users	Prioritize the implementation of personalization features for the software	Add at least two new personalization features for the next sprint. Add a green awareness campaign feature to educate users about sustainability. Add accessibility features for disabled users.
Sprint 5	Developers researched and adopted green coding practices, producing more efficient code. The sustainability representative pushed	Some team members felt overworked because of the tight deadlines	Introduce a system that will monitor the workload of the developers in each sprint to prevent burnout.	Monitor team workload. Add content filtering to detect and block hate comment

Scrum


Sprint Retrospective
Daily Scrum
Sprint Planning
Sprint Backlog
Product Backlog
1 Scrum Team
Increment
Sprint Review
Scrum Framework © 2020 Scrum.org